



Navi 个人导航站

Docker 部署指南

单容器 · 零依赖 · 双栈 · 服务发现 · 图床库 · 可视化编辑 · 密码保护 · 备份恢复

适用版本: Navi 1.x (node:22-alpine 镜像)

文档日期: 2026 年 9 月 19 日

飞牛 fnOS 用户请另见 [docs/fnos-deploy-guide.html](https://docs.fnos-deploy-guide.html) (含 fnOS 专有路径与权限处理)

1. 项目简介

Navi 是一个参考 sun-panel 实现的轻量级个人导航面板，打包为 **All-in-One 单容器镜像**（基于 node:22-alpine，后端零 npm 依赖），一条命令即可完成部署。

- **可视化编辑**：网页上直接增 / 删 / 改 / 拖拽排序导航项，保存即写回服务器
- **内外网切换**：每个站点可分别配置外网、内网地址，一键切换，内网环境自动识别
- **IPv4 / IPv6 双栈**：默认同时监听两种协议栈
- **访问密码保护**：环境变量一键启用，支持「仅密码」或「用户名 + 密码」联合登录，含登录页、会话管理、登录限流
- **数据备份与恢复**：一键导出含校验和的结构化备份文件，导入时校验完整性并二次确认，异常安全拦截
- **卡片图标双来源**：既可从在线图标库（Dashboard Icons / selfh.st / Iconify）选用，也可从**本地图床库**挑选已上传的图片
- **本地图床库**：支持一次性批量上传大量图标（可拖拽），库内可搜索、查看「使用中」引用、批量清理（默认拒绝删除仍被卡片引用的图片）
- **服务发现**：自动识别 Docker 容器 / 本机监听端口，按服务类型匹配图标并生成导航卡片候选（需挂载 Docker Socket）

2. 环境准备

部署前请确保目标机器已安装 Docker（任意主流 Linux 发行版 / Windows / NAS 均可）：

平台	安装方式
Ubuntu / Debian	<code>curl -fsSL https://get.docker.com bash</code>
CentOS / 其他 Linux	参考 Docker 官方文档安装 Docker Engine
Windows / macOS	安装 Docker Desktop 并启动
群晖 / 威联通 NAS	使用 Container Manager / Container Station
飞牛 fnOS	应用中心安装「Docker」应用，首次打开时选定存储空间；命令行操作需在「设置 → 远程访问」中开启 SSH 并 <code>sudo -i</code> 。专有路径与权限处理见 docs/fnos-deploy-guide.html

验证安装：

```
docker --version      # 显示版本号即正常，如 Docker version 27.x
```

本文所有命令在 Linux 终端 / Windows PowerShell 中通用；Windows 用户请把卷挂载路径 `$(pwd)` 替换为绝对路径，例如 `C:\navi\config.json`。

3. 快速部署（三步完成）

1 获取项目文件

将项目文件夹复制到服务器任意目录，并进入该目录：

```
cd /opt/navi # 项目目录，内含 Dockerfile、server.js、public/ 等
```

2 构建镜像

```
docker build -t navi:latest .
```

看到 `Successfully tagged navi:latest` 即构建成功（镜像约 130MB，含完整运行时）。本项目未发布到任何镜像仓库，必须自行构建，或从其它机器 `docker save` / `docker load` 导入。

3 初始化数据目录

```
mkdir -p data/uploads  
cp public/config.example.json data/config.json # 容器内已内置示例，挂载后以此为准
```

4 启动容器（整目录挂载）

```
docker run -d --name navi \  
-p 8080:80 \  
-e NAVI_PASSWORD="改成你的强密码" \  
-e NAVI_CONFIG_PATH=/app/data/config.json \  
-e NAVI_UPLOAD_DIR=/app/data/uploads \  
-v $(pwd)/data:/app/data \  
--restart unless-stopped \  
navi:latest
```

浏览器访问 `http://服务器IP:8080`，输入密码即可进入导航站。

请勿改回单文件挂载（`-v ../config.json:/app/public/config.json`）。写配置采用原子写（先写 `.tmp` 再 `rename` 覆盖），单文件挂载下 `rename` 会跨越两个挂载点，在飞牛 NAS 等平台上会稳定触发 `EBUSY: resource busy or locked`，表现为网页上点「保存」永远失败。整目录挂载后 `rename` 落在同一挂载点内，天然规避该问题。若你此前已用单文件挂载部署，仓库提供一键迁移脚本（见 README「从旧版平滑升级」）。

安全提醒：站点暴露公网前务必设置 `NAVI_PASSWORD` 或 `NAVI_PASSWORD_HASH`，不设密码则任何人都可访问和编辑。推荐同时设置 `NAVI_USERNAME` 启用「用户名 + 密码」联合登录（见第 5 节）。

4. 方式二：docker compose（推荐）

项目自带 `docker-compose.yml`，已预置端口映射、环境变量与**整目录数据挂载**。它通过 `${NAVI_PASSWORD:?...}` 强制校验密码，因此**必须先在项目根目录创建 `.env`**：

```
# 1. 创建密码文件（仓库已提供模板，.env 不会被提交到 Git）
cp .env.example .env
nano .env                                # 把 NAVI_PASSWORD=CHANGE_ME 改成你的强密码

# 2. 初始化数据目录
mkdir -p data/uploads
cp public/config.example.json data/config.json

# 3. 构建并后台启动
docker compose up -d --build

# 4. 查看运行状态
docker compose ps
```

缺少 `.env` 时 `compose` 会直接报错退出（提示 `NAVI_PASSWORD is required`）。这是刻意设计：若改用 `:-` 给默认值，缺变量时会静默传入空密码，导致**认证被关闭、站点变成任何人可访问且可编辑**。

compose 文件关键配置说明：

配置项	说明
<code>ports: "8080:80"</code>	宿主机 8080 → 容器 80，IPv4/IPv6 双栈；可改为其他端口（只改左侧数字）
<code>NAVI_PASSWORD</code>	访问密码，来自 <code>.env</code> （详见第 5 节）
<code>NAVI_USERNAME</code>	登录用户名（可选，设置后启用「用户名 + 密码」联合登录）
<code>SESSION_TTL_HOURS</code>	登录会话有效期（小时），默认 72
<code>SITE_TITLE / SITE_SUBTITLE</code>	覆盖站点标题 / 副标题
<code>volumes: ./data:/app/data</code>	整目录挂载 ： <code>config.json</code> （编辑持久化）与 <code>uploads/</code> （图床库 / Logo 图片）都在其中，切勿拆成单文件挂载
<code>DOCKER_SOCKET + socket 挂载</code>	可选，用于 服务发现 （自动识别容器端口、匹配图标、生成卡片）。需取消注释 <code>- /var/run/docker.sock:/var/run/docker.sock:ro</code> 。见下方安全提示

挂载 Docker Socket 的安全提示：容器内进程将获得等价于宿主机 root 的 Docker 控制权——可读取所有容器的环境变量（常含数据库密码、API Key），也能启动特权容器读写宿主机任意文件。另需注意 `:ro` 只是把 socket 文件标记为只读，**并不能限制通过该 socket 发起的 Docker API 调用**，不要误以为加了它就安全了。因此：仅在你信任镜像来源、且该端口只在内网可访问时开启。不想挂 socket 的话，可改用 `DOCKER_HOST_NAME + DOCKER_HOST_PORT` 连接远程 Docker API，但裸开 2375 端口同样是未加密无认证的；**不开启服务发现也不影响其它任何功能。**

另一种做法：拉预构建镜像（完全不构建）

项目已把镜像发布到 GHCR，因此可以跳过 `docker build`，直接拉取运行。差别只在**换一个编排文件**（两者是二选一的关系）：

编排文件	行为	什么时候用
<code>docker-compose.yml</code>	先 <code>build: .</code> 再起容器	你想改代码、需要重建
<code>docker-compose.image.yml</code>	不含 <code>build:</code> ，只拉 <code>ghcr.io</code> 上的预构建镜像	只是想把服务跑起来

```
# 1. 从镜像里导出初始配置（挂载会遮住镜像内置示例，这步不能省）
docker run --rm ghcr.io/mijunyi/navi-homepage:latest cat /app/data/config.json > data/config.json

# 2. 先 pull 再起（直接 up 会复用本地旧镜像，升级时最容易踩）
docker compose -f docker-compose.image.yml pull
docker compose -f docker-compose.image.yml up -d
```

镜像同时提供 `linux/amd64` 与 `linux/arm64`，Docker 会自动挑对的架构。x86_64 机器命中的是原生的 `linux/amd64`，**不需要加 `--platform`**；但需确认是 64 位（**32 位 i386 无对应镜像**）且 CPU 支持 SSE4.2，详见[拉取镜像部署指南第 6 节](#)。可用 `NAVI_IMAGE`（换镜像站）与 `NAVI_TAG`（固定版本）两个变量覆盖，二者只在解析编排文件时生效、不会注入容器。

完整说明见 [docs/image-deploy-guide.html](#)：包可见性与登录凭据、国内访问 `ghcr.io` 的三种对策、架构匹配、图形界面部署要点、升级回滚与 15 项排查表。

5. 配置访问密码

方式一：明文密码（简单）

```
-e NAVI_PASSWORD="你的强密码"
```

方式二：哈希密码（推荐，环境变量不留明文）

1 在任意装了 Node.js 的机器上生成哈希

```
node -e "console.log(require('crypto').createHash('sha256').update('navi-site:你的强密码').digest('hex'))"
```

2 用哈希启动容器

```
docker run -d --name navi -p 8080:80 \
  -e NAVI_PASSWORD_HASH="上一步输出的64位哈希" \
  -e NAVI_CONFIG_PATH=/app/data/config.json \
  -v $(pwd)/data:/app/data \
  navi:latest
```

注意哈希必须带盐前缀：服务端比对的是 `sha256("navi-site:" + 密码)`。直接用裸 `sha256(密码)` 填进去会永远登录失败（返回 401 且无任何报错，极易误判成密码敲错）。

用 compose 时，把这行写进 `.env` 更省事： `NAVI_PASSWORD_HASH=64位哈希`（并删掉或注释掉 `NAVI_PASSWORD` 那行）。

方式三：用户名 + 密码联合登录（更安全，推荐）

同时设置 `NAVI_USERNAME` 与密码，登录时必须同时输入正确的用户名与密码，两项任一错误均提示「用户名或密码错误」（不区分是哪项，防止用户名被枚举）：

```
docker run -d --name navi -p 8080:80 \
  -e NAVI_USERNAME="admin" \
  -e NAVI_PASSWORD="你的强密码" \
  -e NAVI_CONFIG_PATH=/app/data/config.json \
  -v $(pwd)/data:/app/data \
  navi:latest
```

设置用户名后，登录页会自动多出「用户名」输入框并设为必填。

三种方式的登录体验一致（登录后 72 小时内免重复登录，页头「退出」按钮可随时注销）；密码仅存在于服务器环境变量，代码内无任何硬编码。推荐公网部署采用「方式三 + 方式二的哈希」组合：用户名 + 哈希密码。

6. 自定义导航内容

推荐：直接在网页上编辑——登录后点击右上角「编辑」，即可添加 / 修改 / 删除 / 拖拽排序导航项，管理分组，完成后点「保存」写回服务器。

也可以手动编辑宿主主机上的 `public/config.json`（compose 挂载方式下改完刷新网页即生效）：

```
{
  "title": "Jellyfin",           // 名称 (必填)
  "desc": "影音媒体库",        // 描述 (可选)
  "icon": "jellyfin",           // 图标 (可选)
  "url": "https://media.example.com", // 外网地址 (必填)
  "lanUrl": "http://192.168.1.10:8096" // 内网地址 (可选)
}
```

icon 写法	图标来源
"jellyfin"	Dashboard Icons (按名称自动匹配, 推荐首选)
"selfhst:portainer"	selfh.st 图标库
"iconify:simple-icons:github"	Iconify 全量图标库
"https://.../logo.png"	任意图片直链
留空	按名称尝试 Dashboard Icons, 失败回退首字母图标

卡片图标的两种来源

编辑导航项的弹窗里, 图标区分为两行: 「**Logo (本地图床库)**」与「**在线图标**」。两者可同时存在, 渲染优先级为 **本地图床库 > 在线图标 > 字母回退**。

来源	存储字段	说明
本地图床库	logo	存为 <code>/uploads/<文件名></code> , 实体位于宿主机挂载目录的 <code>uploads/</code>
在线图标	icon	写法见上表 (<code>jellyfin</code> / <code>selfhst:xxx</code> / <code>iconify:集:名</code> / 图片直链)

本地图床库 (批量上传与复用)

编辑模式底部点「图床库」打开弹窗, 分为**本地图床库**与**在线图标库**两个标签页:

- **批量上传**: 一次可多选或**直接拖入**大量图片, 支持 **PNG / JPEG / GIF / WebP**, 单张上限 **3MB**; 前端按「字节预算 + 单批 20 张」自动分片上传并显示进度
- **逐张独立校验**: 服务端按**文件头魔数**识别真实类型, 防止伪造图片; **个别失败不影响其余图片**, 逐条给出失败原因
- **重复使用**: 上传的图片不再是「用完即弃」, 任何导航卡片都可在图床库里挑选复用
- **检索与引用可见**: 按文件名实时过滤, 标注「使用中 N」并显示引用它的卡片名
- **删除保护**: 仍被卡片引用的图片**默认拒绝删除** (返回 409 + 引用清单), 二次确认后才强制删除, 避免线上卡片集体变成字母图标
- **存储位置**: 文件落在 `NAVI_UPLOAD_DIR` (Docker 即 `/app/data/uploads`), 通过 `/uploads/<文件名>` 访问。务必持久化该目录, 否则容器重建后图片全部丢失

图床库不建数据库、不写索引文件：库内容直接由上传目录的文件系统推导，因此永远与磁盘一致，也不会给备份/恢复流程引入额外状态。另注意网页上的「备份」按钮**只导出 config.json**，**不打包图片实体**；迁移时请一并复制 uploads/ 目录（图片缺失时卡片会自动回退字母图标，不会报错，所以容易察觉不到）。

7. 数据备份与恢复

编辑模式底部提供「备份」与「导入」两个按钮，用于整体迁移或防误删：

备份（导出）

点击「备份」下载结构化备份文件 `navi-backup-<时间>.json`，内含格式标识、版本、导出时间、完整导航配置，以及基于配置计算的 **SHA-256 校验和**，可跨设备、跨部署迁移。

恢复（导入）

点击「导入」选择备份文件后，系统会依次校验 **文件格式** → **版本** → **完整性（校验和）** → **数据结构**，全部通过后弹出二次确认（显示将覆盖的导航项数量），确认后才原子写回并自动刷新。

异常安全：文件损坏、格式不匹配、版本过高、校验和篡改、结构非法等任一情况都会被拦截并给出明确报错，**绝不会破坏现有数据**——只有全部校验通过才会写入。

备份文件是纯 JSON，可安全存放在本地或网盘；恢复后数据与服务器当前状态完全一致。

关于「完整性校验失败」：导入时的校验和比对是**以后端为准**的权威校验。前端只在浏览器支持的情况下做一次「提前预检」——浏览器仅在**安全上下文**（HTTPS 或 localhost）才提供 `crypto.subtle`，所以用 `http://` 局域网 IP: 端口 访问时会自动跳过预检、直接交给后端，**不会影响导入，也不会削弱篡改防护**。若导入自己刚导出的文件仍报此错，说明文件确实被改动过（例如用编辑器打开后保存、传输过程损坏），重新导出一份即可。

8. IPv6 访问

容器内部默认双栈监听（`::`），宿主机开启 IPv6 时 Docker 端口映射会同时绑定两种协议栈，无需额外配置：

```
# 验证容器双栈监听
docker exec navi netstat -tlnp      # 应看到 :::80

# 通过 IPv6 地址访问
curl -g "http://[2001:db8::1]:8080/api/health"
```

若 Docker 未自动绑定 IPv6，请在宿主机 `/etc/docker/daemon.json` 中设置 `"ipv6": true` 与 `"fixed-cidr-v6"` 后重启 Docker。

9. 日常运维命令

操作	命令
查看日志	<code>docker logs -f navi</code>
停止 / 启动 / 重启	<code>docker stop navi · docker start navi · docker restart navi</code>
修改密码 / 用户名	改环境变量后重建容器（见下方更新流程）
备份导航数据	网页编辑模式点「备份」下载备份文件；或直接复制宿主机 <code>public/config.json</code> 与 <code>public/uploads/</code>
删除容器与镜像	<code>docker rm -f navi && docker rmi navi</code>

升级到新版项目代码

```
# 配置在宿主机挂载，重建不丢数据
docker compose up -d --build      # compose 方式

# 或纯 docker 方式：
docker rm -f navi
docker build -t navi .
docker run -d --name navi # ...与初次启动相同的参数
```

服务重启后所有登录会话会失效（会话密钥每次启动随机生成），用户重新登录即可，属正常安全设计。

10. 部署验证

1 健康检查

```
curl http://127.0.0.1:8080/api/health
# 期望输出: {"ok":true,"ipv6":true,"authEnabled":true}
```

2 页面访问

浏览器打开 `http://服务器IP:8080` → 应跳转登录页 → 输入用户名 / 密码 → 看到导航卡片。

3 自动化测试（可选）

项目内置 **940 项断言 / 19 个套件**。最省事的方式是一条命令全跑（它会自动用「临时 config + 临时 uploads」在 8633 端口拉起隔离实例，不碰真实数据）：

```
NODE_PATH=<已装 playwright 的 node_modules> node test/run-all.cjs
```

排查单个功能时按文件名过滤，只跑关心的套件

```
NODE_PATH=<已装 playwright 的 node_modules> node test/run-all.cjs ui-backup
```

也可以按需单独运行：

① 自包含套件（内部自行拉起隔离实例，直接运行即可）

node test/auth.test.js	# 密码保护 / 联合登录	27 项
node test/backup.test.js	# 备份恢复 / 图片魔数校验	24 项
node test/zipbackup.test.js	# 含图片的 zip 备份	54 项
node test/status.test.js	# 状态板（伪造 Docker API）	90 项
node test/discover.test.js	# 服务发现（伪造 Docker API）	99 项
node test/library.test.js	# 图床库（引用保护 / 批量上传）	41 项
node test/icons.test.js	# 本地图标库自检	47 项
node test/ui-auth.test.cjs	# 登录页 UI	8 项
node test/ui-discover.test.cjs	# 服务发现 UI	58 项
node test/ui-library.test.cjs	# 图床库 UI	47 项
node test/ui-firstrun.test.cjs	# 首次引导 / 拖链接建卡 UI	33 项
node test/ui-tags.test.cjs	# 卡片标签 / 跨分组检索 UI	40 项
node test/checkdeploy.test.cjs	# 部署判定探针自检	22 项
node test/imagecompose.test.cjs	# 镜像发布契约自检	125 项

② 需要先有一个运行中的实例（下面五条都要传 baseUrl）

node test/server.test.js	http://127.0.0.1:8080	# 服务端集成	50 项
node test/ui.test.cjs	http://127.0.0.1:8080	# 主界面 UI	103 项
node test/ui-theme.test.cjs	http://127.0.0.1:8080	# 主题切换 UI	8 项
node test/ui-backup.test.cjs	http://127.0.0.1:8080	# 备份 UI	32 项
node test/ui-status.test.cjs	http://127.0.0.1:8080	# 状态板 UI	32 项

跑 ② 那五条前请先建一个隔离实例（把配置与上传目录指向临时位置），不要直接对生产实例执行——

`ui.test.cjs` 会真实保存配置、`ui-backup.test.cjs` 会临时改写并还原 `public/config.json`。示例：

```
PORT=8633 NAVI_CONFIG_PATH=/tmp/navi-test/config.json \
  NAVI_UPLOAD_DIR=/tmp/navi-test/uploads node server.js
# 另开一个终端再跑测试，URL 用 http://127.0.0.1:8633
```

浏览器端 UI 测试复用本机 Edge/Chrome 内核，无需下载 Chromium；需先设置 `NODE_PATH` 指向已安装 playwright 的目录。

注意区分「正常拒绝」与「部署失败」：启用密码保护后，`/api/config` 等接口在未登录时返回 401、页面返回 302 跳登录页——这是安全设计生效的表现。判断服务是否正常，只看 `/api/health` 是否返回 200。

改了源码、重建了镜像，页面却还是老样子？先跑一次部署判定探针，它会明确告诉你容器里跑的是不是新代码：

```
node scripts/check-deploy.cjs http://服务器IP:8080 你的密码
# 退出码 0 = 新代码；1 = 仍是旧代码；2 = 检查无法完成（如端口不通/密码错）
# 密码不对或没给密码时不会中断：会凭公开的 /login.html 响应头给 server.js 定性，
# 并标注 app.js 那一项「未验证」——部分判定，绝不假装通过
```

它检查 HTML 的缓存策略、前端资源版本号是否为服务端自动推导、以及线上 `app.js` 是否含最新内容。若是旧代码，通常是漏传了某个文件（`server.js` / `discovery.js` / `public/` 缺一不可）或浏览器仍用 7 天强缓存的旧 `js/css`。脚本最后还会打印容器内文件的比对命令与期望字节数，照着 `wc -c` 出一遍就能确认源码是否同步。